

4.5 Algorithm of PrismSpace Multi-Agent System

The PrismSpace algorithm orchestrates multiple specialized AI agents into a coordinated swarm: interpreting user objectives, routing tasks to the optimal LLM provider, dispatching specialized agents, equipping them with tools, and verifying safety and accuracy. The workflow operates in three distinct phases:

- **Phase I: Offline Swarm Graph Representation** (Steps 1–5: Data prep, swarm graph, GAT routing)
- **Phase II: Single-Stage Preference Alignment** (Step 6: Safety and quality tuning via ORPO)
- **Phase III: Online Runtime Inference & Swarm Execution** (Steps 7–9: Live execution, evaluation, telemetry)

Inputs (What Goes In):

- **User Objective** ($\mathcal{T}_{\text{user}}$): Natural language user prompt, operational scope, and execution constraints.
- **Curated Training Corpora** ($\mathcal{D}$): Multi-source benchmark datasets containing prompt traces, intent categories, safety evaluations (WildGuard), and multi-agent trajectories (RouterBench, LMSYS Arena, CMU Agent Benchmark).
- **Preference Alignment Pairs** ($\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$): Prompt x paired with preferred response y_w and disfavored response y_l .
- **Global Agent Swarm Registry** ($\mathcal{G}_{\text{global}} = (V, E)$): Roster of specialized AI agents (Supervisor, Coding, Research, Validator, Planner) and Model Context Protocol (MCP) tool endpoints.

Outputs (What Comes Out):

- **Task Intent & Best Provider**: Predicted task category ($\hat{y}_{\text{intent}}$) and optimal LLM provider ($\hat{y}_{\text{provider}}$, e.g., Groq for rapid response, OpenAI/Anthropic for complex reasoning).
- **Swarm Dispatch Vector** ($\hat{y}_{\text{agents}}$): Binary activation checklist (1 = active, 0 = idle) for each agent in V .
- **Execution Plan** ($\mathcal{G}_{\text{active}}$): Directed Acyclic Graph (DAG) outlining task steps with predicted latency ($\hat{\ell}$) and cost ($\hat{c}$).
- **Tuned Model Adapters** (ΔW): Lightweight LoRA weights ensuring helpfulness and safety compliance.
- **Evaluation Metrics**: Quantitative benchmarks including Accuracy, F1-Score, ROC-AUC, and ECE calibration.

Step 1: Data collection and Preprocessing

1. Import and unify raw records across multi-source benchmark datasets ($\mathcal{D}$), mapping heterogeneous prompt schemas, classification labels, and tool invocation logs into a structured, unified schema.
2. Clean and sanitize the textual dataset by stripping malformed Unicode sequences, removing redundant whitespace tokens, trimming uninformative control characters, and filtering duplicate prompt-response pairs.
3. Perform categorical encoding across discrete metadata attributes (agent role definitions, tool protocol capabilities, provider endpoints) into dense numerical vectors.
4. Standardize continuous numerical features (token length, baseline query complexity, empirical latency traces) using z -score normalization: subtract the empirical feature mean and divide by standard deviation ($z = (x - \mu)/\sigma$) to yield zero mean and unit variance across all dimensions.
5. Partition the processed dataset via stratified random sampling into three non-overlapping subsets:
 - * 80% Training set ($\mathcal{D}_{\text{train}}$) for model parameter optimization and graph embedding learning
 - * 10% Validation set ($\mathcal{D}_{\text{val}}$) for hyperparameter tuning, pruning, and early stopping
 - * 10% Testing set ($\mathcal{D}_{\text{test}}$) for unbiased evaluation of routing accuracy and safety alignment

Step 2: Swarm Graph Construction

1. Define the node set (V): represent each specialized AI agent persona (Supervisor, Research, Coding, Validator, Planner) and each Model Context Protocol (MCP) tool as an individual vertex $v_i \in V$.
2. Evaluate pairwise functional capability similarity: compute the Pearson correlation coefficient between node capability vectors $\mathbf{x}_i$ and $\mathbf{x}_j$, defined as the covariance of their feature distributions divided by the product of their standard deviations ($s_{ij} = \text{Cov}(\mathbf{x}_i, \mathbf{x}_j) / (\sigma_i \sigma_j)$).
3. Establish collaborative topological connections: generate binary adjacency matrix $\mathbf{A}$ by comparing similarity against correlation cutoff τ , setting $A_{ij} = 1$ when pairwise similarity meets or exceeds threshold ($s_{ij} \geq \tau$) and $A_{ij} = 0$ otherwise.
4. Construct the global swarm topology $\mathcal{G} = (V, E)$: establish undirected collaborative communication edges E that permit message propagation exclusively between complementary agent and tool pairings.

Step 3: Graph Attention Network (GAT) Initialization

1. Initialize node representations: assign each vertex $v_i \in V$ an initial d -dimensional feature embedding vector $\mathbf{h}_i = \mathbf{x}_i \in \mathbb{R}^d$ describing agent operational scope, prompt semantics, and tool interface signatures.
2. Initialize projection and attention parameters: instantiate weight projection matrix $\mathbf{W} \in \mathbb{R}^{d' \times d}$ and shared attention mechanism vector $\mathbf{a} \in \mathbb{R}^{2d'}$ via Xavier uniform initialization, ensuring stable variance across forward and backward signal propagation.
3. Configure network hyperparameters: establish learning rate η , attention head count K (specialized across task intent, model capability, swarm affinity, and safety constraints), and dropout regularization probability p .

Step 4: Attention-Based Message Passing

For each node v_i with immediate collaborative neighborhood $\mathcal{N}(i)$:

1. Compute raw attention coefficients with connected neighbors $v_j \in \mathcal{N}(i)$: project node features through transformation matrix $\mathbf{W}$, concatenate projected states $[\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]$, compute inner product with attention vector $\mathbf{a}$, and apply LeakyReLU non-linearity: $e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])$.
2. Normalize coefficients across the local neighborhood using Softmax: calculate positive attention weights $\alpha_{ij} = \exp(e_{ij}) / \sum_{k \in \mathcal{N}(i)} \exp(e_{ik})$ that sum to unity, quantifying the relative communicative significance of neighbor j to node i .
3. Aggregate neighborhood feature representations: compute a weighted linear sum of neighbors' transformed representations scaled by attention weights, followed by non-linear activation $\mathbf{h}'_i = \sigma(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j)$.
4. Multi-head attention concatenation: execute K independent attention heads in parallel and concatenate their output representations $\mathbf{h}'_i = \parallel_{k=1}^K \sigma(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j)$, capturing rich, multi-perspective collaborative semantics.

Step 5: Classification and Loss Computation

1. Route updated swarm representations $\mathbf{h}'_i$ through dual specialized prediction heads:
 - * *Intent & Provider Routing Head*: Computes normalized routing probabilities via Softmax ($\hat{\mathbf{y}}_{\text{route}} = \text{Softmax}(\mathbf{W}_{\text{route}} \mathbf{h}' + \mathbf{b}_{\text{route}})$), selecting the optimal foundation model provider based on task difficulty and speed.
 - * *Agent Dispatch Head*: Evaluates independent activation probabilities per agent persona via element-wise Sigmoid ($\hat{y}_{\text{agent},i} = \text{Sigmoid}(\mathbf{W}_{\text{agent}} \mathbf{h}'_i + b_i)$), determining active swarm membership.
2. Compute unified multi-task loss ($\mathcal{L}_{\text{class}}$): combine categorical cross-entropy for provider routing with binary cross-entropy across agent dispatch activations, balancing classification accuracy with minimal swarm over-provisioning: $\mathcal{L}_{\text{class}} = \mathcal{L}_{\text{CE}}(\mathbf{y}_{\text{route}}, \hat{\mathbf{y}}_{\text{route}}) + \lambda \sum_{v \in V} \mathcal{L}_{\text{BCE}}(y_v, \hat{y}_v)$.
3. Optimize parameters: backpropagate error gradients and update network parameters using the AdamW optimizer with cosine learning-rate decay and gradient norm clipping.

Step 6: Single-Stage Preference Alignment (ORPO)

1. Inject Parameter-Efficient Fine-Tuning (PEFT) adapters: attach low-rank decomposition matrices ($\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{BA}$) to attention projection layers of the base language model, enabling targeted adaptation while keeping foundational weights frozen.
2. Compute the supervised fine-tuning (SFT) objective: calculate the sequence negative log-likelihood loss exclusively on favored human responses (y_w), preserving foundational language fluency and structured syntax: $\mathcal{L}_{\text{SFT}} = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P(y_{w,t} \mid x, y_{w,<t})$.
3. Calculate length-normalized sequence odds: evaluate the likelihood of token sequence $y \in \{y_w, y_l\}$ using the odds formulation $\text{odds}(y \mid x) = P(y \mid x) / [1 - P(y \mid x)]$, where per-token probabilities are geometrically averaged across sequence length to eliminate bias favoring shorter answers.
4. Formulate the Odds Ratio (OR) preference penalty: calculate the negative log-sigmoid of the log-odds ratio between favored (y_w) and disfavored (y_l) completions: $\mathcal{L}_{\text{OR}} = -\log \sigma(\log[\text{odds}(y_w \mid x) / \text{odds}(y_l \mid x)])$, actively disincentivizing unsafe, hallucinated, or low-quality patterns.
5. Optimize unified single-stage ORPO objective: jointly minimize the composite loss $\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \beta \cdot \mathcal{L}_{\text{OR}}$, achieving concurrent instruction following and safety preference alignment without requiring a separate reward model or reinforcement learning phase.

Step 7: Real-Time Runtime Inference and Swarm Execution

1. Capture user objective: ingest natural language prompt $\mathcal{T}_{\text{user}}$ via the Next.js client interface and securely dispatch it to the FastAPI Hive Bridge orchestration gateway.
2. Synthesize query feature vector ($\mathbf{x}_{\text{query}}$): concatenate sparse TF-IDF n-gram vectors, dense semantic sentence embeddings, and session contextual metadata (token budget, latency requirements, user tool permissions).
3. Determine routing and dispatch policy: assign the prompt to the highest-scoring LLM provider via argmax selection on routing probabilities ($\hat{y}_{\text{provider}} = \text{argmax}(\hat{\mathbf{y}}_{\text{route}})$), and activate all worker agents whose dispatch score meets threshold τ_{dispatch} ($\hat{\mathbf{y}}_{\text{agents}} = \mathbb{I}(\hat{\mathbf{y}}_{\text{dispatch}} \geq \tau_{\text{dispatch}})$).
4. Formulate task DAG and execution schedule: generate active dependency graph $\mathcal{G}_{\text{active}}$ and execute a topological sort ($\pi = \text{TopologicalSort}(\mathcal{G}_{\text{active}})$) to determine parallel and sequential task execution phases.
5. Execute autonomous ReAct swarm loops: dispatch agents to iteratively reason, propose actions, and invoke Model Context Protocol (MCP) tool endpoints (Browser, Filesystem, GitHub, Python Sandbox), aggregating intermediate tool observation traces until all subtask criteria are satisfied.
6. Enforce policy safety gating: evaluate final candidate completions through the calibrated safety gate, verifying that the log-odds safety score relative to reference text meets safety threshold τ_{safety} , intercepting prompt injection, unsafe operations, or policy violations.
7. Deliver real-time response and persist session state: stream verified response tokens live to the frontend via Server-Sent Events (SSE), update real-time telemetry HUD metrics, and commit complete conversation artifacts to local browser IndexedDB storage using Dexie.js.

Step 8: Model Evaluation

1. Assess generalization performance: benchmark trained GAT routing and ORPO alignment models against holdout validation ($\mathcal{D}_{\text{val}}$) and test ($\mathcal{D}_{\text{test}}$) splits to quantify accuracy and preference adherence.
2. Compute comprehensive quantitative validation metrics:
 - * *Routing Accuracy & Macro-F1*: Measure proportion of correctly classified intents and provider selections, computing harmonic mean of precision and recall to evaluate balanced performance across rare and frequent task classes.
 - * *Discrimination Capability (ROC-AUC & AUPRC)*: Measure the area under the receiver operating characteristic and precision–recall curves to assess safety gate discrimination under class imbalance.
 - * *Calibration Quality (ECE)*: Measure Expected Calibration Error across M probability bins ($\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$) to verify that predicted routing confidence aligns with empirical accuracy.
 - * *Regression Precision (R^2 & MAE)*: Quantify predictive fidelity of task latency ($\hat{\ell}$) and monetary token cost ($\hat{c}$) against actual multi-agent execution traces.
3. Save optimized model checkpoints, export weight artifacts, and log diagnostic performance reports.

Step 9: Visualization and Interpretation

- Generate convergence diagnostics: plot training and validation loss trajectories, ROC curves, and precision–recall frontiers to track learning progression and over-fitting boundaries.
- Inspect attention heatmaps: visualize inter-agent and agent-tool attention weight matrices across the K attention heads, identifying dominant collaboration pathways and tool utilization dependencies.
- Monitor live operational telemetry: track real-time routing hit rates, token cost allocations ($\hat{c}$), task latency profiles ($\hat{\ell}$), and safety policy pass rates on the Swarm Telemetry HUD.

Algorithm Summary

1. **Graph-Attentive Swarm Routing**: The GAT architecture models functional interactions between specialized agents and MCP tools, dynamically routing incoming tasks to the optimal LLM provider and dispatching minimal required worker agents.
2. **Single-Stage Preference Alignment**: The ORPO formulation unifies supervised fine-tuning with length-normalized odds ratio penalties, tuning model helpfulness and safety in a single stage without independent reward models.
3. **Autonomous Reactive Execution**: Runtime execution is scheduled via DAG topological sorting, iterative ReAct tool loops, rigorous odds-ratio safety gating, and live SSE streaming to browser IndexedDB storage.